

ABRA — where things stand, 2026-07-28 evening

Everything here comes from a run. Where a number is missing it is marked pending, not guessed.

The short version

Three questions got answered today, and **two of my own conclusions were wrong before they were right**. Both were caught by counters and cross-checks rather than by results, which is the part worth keeping.

1. **The forfeit hypothesis is dead.** I told you the forfeit corpus was probably why your mechanics work looked flat. It is not. Two independent measurements say so.
2. **DODUO — the joint layer — is wired in, measured, and it LOSES.** It had never once been in the loop. Coordination as fitted by imitation wins 28.4% [23.9, 33.3] of decisive pairs against the same bot with the coordination weights zeroed. It stays off by default. This does NOT settle your exploitability argument, which is a different question and still open.
3. **MACHAMP produced a champion that beats the imitation-fitted MAG 57.3%.** Shipping it is your call, not mine.

1. Forfeits — my hypothesis was wrong

I said refitting without forfeited games might rescue the mechanics batch. It does not.

held-out set	with forfeits	without	difference
completed games	31.24%	31.36%	+0.12
forfeited games	28.15%	28.01%	−0.14
both	30.41%	30.46%	+0.05

And in actual play, 1,365 decided games: the no-forfeit vector wins **48.6% [46.0, 51.3]**. Straddles 50, and holds on both sides of the board (47.4% on p1, 49.9% on p2).

Prediction and play agree. Removing forfeited games from the fit changes nothing. Your mechanics ideas are still unexplained, and the corpus is not the reason.

Not done: that head-to-head was not seed-paired. `mew.js --paired` exists and I did not use it, so the interval is wider than it needed to be. The verdict is not close enough to 50 for pairing to change it.

2. DODUO — the joint layer (named today)

Two heads, one body: two slots, one decision. It scores the PAIR of choices rather than two choices separately — 18 coordination features (focus fire, redirect-then-attack, Helping Hand into a kill, Tailwind that flips the partner's speed order, and so on).

Why it was not retired

I retired it earlier on the double-target rate — MAG 24.6% against humans 23.2%. That metric touches **2 of the 18 features**. Your argument is what reversed it: a bot that picks each slot independently can be set positions that REQUIRE coordination and will fail them every time. That is a repeatable hole, not variance.

Refitted at 48 features

5,250 clean games, 18,740 usable joint turns, 15,345 train / 3,395 held out:

predicting which PAIR a human clicked	log-lik	top-1
two moves decided separately (what MAG did)	-3.6374	5.9%
refitted, coordination terms forced to zero	-3.1643	12.0%
with the coordination terms	-2.9890	14.5%

Read the middle row before the last one. Over half the gain is just refitting the ordinary move weights on pair data. Only the final 2.5 points belong to coordination itself. And all of this predicts a human click — it says nothing about winning.

Three bugs found while wiring it, all in my own work

a. It was doing nothing, silently. First smoke test: **0 pairs decided, 99 fallbacks to independent choice**. No error, no discarded games, run looked clean. The partner's move list was read from the raw request ({move:'Protect', id, pp, target, disabled}) instead of the reshaped list chooseMove actually receives ({choice, move:{slot, move, target}}), so every partner option parsed as unusable.

Had I gone straight to the head-to-head it would have returned ~50% and I would have reported that coordination does not help — a false negative on your argument. It was caught by a **counter that prints how often the pair path bails**, not by any result. Now 99.5% of eligible turns are decided as a pair.

b. The pair score summed two different fits. fit_joint.js fits its move weights and its coordination weights *together*, so the coordination terms are calibrated against *its* move weights. I scored the moves with the **shipped** weights and added the joint fit's coordination terms on top. Those two sets disagree badly:

- **23 of 48 features carry opposite signs**
- stallIntoEncore is **+4.231** in the joint fit against **-1.993** shipped
- vector norms 10.07 against 7.45

That first head-to-head measured the hybrid at **42.7% [40.5, 44.9]** over 1,942 games and **31.2% [26.8, 36.1]** on 378 decisive pairs. The number is real. It does not answer the question it appears to answer, and it is **not evidence against your argument**.

c. There was no control. Comparing "joint on" to the ordinary bot changes three things at once — coordination terms, move weights, and the fact that it now picks from a shortlist of PAIRS rather than per slot. That last one changes behaviour on its own with every coordination weight set to zero, and it would have been credited to coordination. --joint-zero now runs the entire pair path with the 18 coordination weights zeroed.

The corrected experiment — coordination LOSES

`--joint` against `--joint-zero2`, 2,000 seed-paired games, identical in every other respect. Harness fair at 49.3% [47.1, 51.6].

	result
unpaired win rate, coordination ON	42.0% [39.9, 44.3] over 1,934 games
decisive pairs, coordination ON wins both	28.4% [23.9, 33.3] of 356

Not close, well powered, consistent in every cut. Worst in short games (22.0% under 8 turns) and when it does not draw first blood (14.1%) — a bot giving away tempo. It KO's less (22.3% against 25.1%) and Protects nearly twice as often (1.74% against 0.93%).

Why: these are imitation weights. The fit prices `spreadFreeBesideAlly` at -5.054 , `terrainSetupHelpsPartner` at -3.989 and `screenWhileThreatened` at -3.372 , at $\lambda = 0$. Those say humans rarely click those pairs, not that the pairs are bad. A bot told to avoid a free spread move beside its own ally by -5 declines its best plays. Same lesson MACHAMP taught, and the cleanest separation of predicting from winning the project has measured: 14.5% top-1 on human pairs, 28.4% of decisive pairs won.

(The earlier 31.2% belongs to the scrambled hybrid in (b). Do not quote it. It happens to land near this figure, which is a coincidence — it was measuring a different thing.)

What it still does not test

Your claim was about **exploitability**, not average win rate. Even a coordinated bot that wins no more often could be harder to counter, and that is the thing you said matters. `exploit.js` now takes `--target <weights.json>`, so WOBBUFFET can be pointed at a specific vector for the first time. Not yet run.

3. MACHAMP — a champion exists

Ran on 48 features with the annealing fixed. Gen 1 promoted.

- confirmation **55.7%** $n=393$ [50.8, 60.6]
- against gen 0 (imitation-fitted MAG) **57.3%** $n=199$ [50.3, 64.0], no cycle detected

First positive evidence for the win objective over the imitation objective. The champion is recorded in `data/ladder.json` and exported as a loadable 48-weight file.

The annealing bug: the old schedule shrank the step $0.7 \rightarrow 0.184$ over 8 generations while n stayed at 400, so the Wilson promotion gate could never clear — it was guaranteed to refuse forever. `SHRINK` now defaults to 1 and `STOP_AFTER=3` consecutive failures at full scale replaces the `--gens 8` I had invented with no criterion behind it.

Shipping it is your call. Measuring is mine; retiring, deleting and promoting are yours.

4. Tooling fixes that prevent silent failures

- A same-name A/B was unscorable and reported nothing rather than failing.**

`winnerPolicy` records a policy NAME, and the A/B this project runs most often is `--policy score --`

`policy2 score` with two weight files. All 1,365 games came back labelled "score", so every pair scored as a tie and `h2h_stats` printed zero decisive pairs in every cut instead of saying it could not tell the arms apart. `mew.js` now writes `winnerArm` and `winnerWeights`.

- **A config error was indistinguishable from a battle failure.** Requesting the joint layer with a stale weight file printed "4 discarded" and nothing else; the thrown message naming the actual problem never reached the terminal.
 - `armsIdentical` **claimed a mirror match during the actual experiment** — it checked the policy name and weights file only, so a `--joint` A/B (which differs by a flag) was stamped as a mirror.
-

Open, in the order I would take them

1. **WOBBUFFET on `--target`** — exploitability of MACHAMP's champion, and of DODUO. This is the test that actually carries your argument, and it has never been run against anything but the shipped imitation vector.
 2. **Refit the 18 coordination weights for WINNING, not for resemblance** — MACHAMP over the joint vector. The coordination FEATURES are not refuted by the above; the imitation fit of them is. This is the untested version of the idea and it follows directly from the result.
 3. **Re-run what was computed on the pre-forfeit corpus** — partially done. `nmf-roles` deliberately not regenerated at the unsupported rank 6.
 4. **SLOWKING's intervals** — diagnosed, unfixed, your call. The bootstrap solves at 1,000 iterations while the point estimate solves at 15,000, so the interval is measuring solver convergence error rather than uncertainty.
 5. **Transform awareness for Ditto and Zoroark** — megas are done, these are not.
 6. **Defensive type chart** — MAG still cannot see what hits IT.
 7. **Volatiles on the field** — needs ingest first or the features compute to constant zero.
 8. **Play MAG yourself** — deferred by you until it feels ready.
-

Standing rules I am holding to

- Measured, not asserted. Negative results reported plainly and in the same detail as positive ones.
- Measuring is mine. Retiring, deleting, disabling and shipping are yours.
- Plain numbers and percents.
- Never baseline against the raw ladder store — everything behavioural goes through `engine/quality.js`.
- Never edit `board.js` while a fit or self-play run is in flight.
- Commit and push in the same pass as any change.